Sentiment Classification and Temporal Analysis under Processing Constraints

An empirical study using TweetEval and Bluesky

Submitted in partial fulfillment of the requirements of the degree of

M.Sc. Data Science and AI

[Learner full name]

SAP number [To be supplied]

Guide [Name to be supplied]

SVKM’s Mithibai College of Arts, Chauhan Institute of Science and Amrutben Jivanlal College of Commerce and Economics (Autonomous)

Department [Official department name]

Academic year [To be confirmed]

Abstract

This study evaluates sentiment classification and temporal aggregation under local processing constraints. VADER, a TF-IDF logistic-regression baseline, a bidirectional LSTM and a Twitter RoBERTa checkpoint are compared on 12,284 labeled TweetEval test examples. RoBERTa achieves 72.18% accuracy and 0.724 macro F1, compared with 59.47% and 0.558 for the saved BiLSTM predictions, 59.22% and 0.560 for the linear baseline, and 53.00% and 0.529 for VADER. A separate corpus of 127,741 Bluesky posts returned by an AI search supports temporal analysis over fourteen days. VADER and RoBERTa produce opposite net-sentiment signs in 327 of 336 hourly periods. Repeated CPU inference measurements and a controlled replay through a bounded queue and SQLite storage characterize processing costs and overload behaviour. The evidence shows that classifier selection changes both benchmark performance and the interpretation of aggregate sentiment. Bluesky correctness remains unmeasured because no independent labels are available. The study contributes a reproducible empirical evaluation with explicit sampling, execution and inference boundaries.

Keywords — sentiment analysis, TweetEval, Bluesky, RoBERTa, BiLSTM, processing latency, temporal aggregation

Contents

1 Introduction 1

1.1 Background 1

1.2 Problem statement 1

1.3 Objectives and research questions 1

1.4 Scope and contribution 2

2 Review of Related Literature 3

2.1 Lexicon and recurrent approaches 3

2.2 Transformers and domain-specific checkpoints 3

2.3 Benchmarking and domain transfer 3

2.4 Existing Bluesky systems and temporal data 4

2.5 Research gap addressed and its limits 4

3 Research Methodology 5

3.1 Study design 5

3.2 Collection and sampling 5

3.3 Model configurations 6

3.4 Metrics and uncertainty 6

3.5 Temporal analysis 7

3.6 Runtime and replay protocols 7

4 System Implementation 8

4.1 Data paths 8

4.2 Storage and reproducibility 8

4.3 Verification 9

5 Results and Discussion 10

5.1 Labeled benchmark performance 10

5.2 Errors and paired comparisons 10

5.3 Historical sentiment and agreement 11

5.4 Temporal interpretation and coverage 12

5.5 Warm inference measurements 12

5.6 Controlled replay and overload 13
5.7 Answers to the research questions 13
6 Conclusion and Future Scope 14
6.1 Conclusion 14
6.2 Limitations 14
6.3 Future scope 14
References 15
Appendix A Reproduction and Evidence 16
A.1 Execution order 16
A.2 Report assembly notes 16

List of Tables

Abbreviations used in this dissertation vi

Table 2.1 Relationship between selected prior work and this study 4

Table 3.1 Datasets and their roles 5

Table 4.1 Implementation components 8

Table 5.1 Common labeled test performance and bootstrap accuracy intervals 10

Table 5.2 RoBERTa confusion matrix 11

Table 5.3 Historical class assignments 11

Table 5.4 Historical pairwise agreement 11

Table 5.5 Sensitivity to aggregation interval for VADER and RoBERTa 12

Table 5.6 Controlled replay over three runs per condition 13

List of Figures

Figure 5.1 Accuracy and macro F1 on the local TweetEval test split. Source — benchmark prediction files. 10

Figure 5.2 Six-hour trailing mean net sentiment and hourly cap-hit requests. Source — historical temporal exports, 19 October–1 November 2026 UTC. 12

Abbreviations and Notation

Abbreviations used in this dissertation

Term	**Meaning**
API	Application programming interface
BiLSTM	Bidirectional long short-term memory
CPU	Central processing unit
F1	Harmonic mean of precision and recall
LR	Logistic regression
NLP	Natural language processing
TF-IDF	Term frequency–inverse document frequency
UTC	Coordinated Universal Time
κ	Cohen’s kappa
p50 / p95 / p99	50th, 95th and 99th latency percentiles
S(t)	Net sentiment in period t
n(t)	Number of observed posts in period t

1 Introduction

1.1 Background

Social-media sentiment monitoring combines text classification with a process that receives, filters and stores observations over time. A useful monitor must produce interpretable sentiment estimates while keeping up with incoming posts. The choice of model affects the labels, computation cost and the temporal signal derived from those labels. Consequently, a classifier with a high benchmark score can still require explicit engineering decisions about queueing and throughput.

Bluesky provides public data through the AT Protocol ecosystem [9]. This project uses Bluesky because it offers an accessible collection interface. The research question concerns sentiment analysis and processing behaviour. Platform accessibility motivates the data source, while the study's empirical findings remain conditional on the collected sample.

1.2 Problem statement

The supplied project proposal called for a comparison of sentiment models under real-time constraints, including accuracy, latency, throughput and temporal analysis. A comparison limited to class counts would leave correctness and runtime behaviour unresolved. This study therefore combines a labeled benchmark, an unlabeled historical deployment sample and controlled processing experiments. Each source answers a different empirical question.

The project requires a reliable distinction between a model's correctness on labeled examples, agreement between models on common posts, and the system's ability to process arrivals. Mixing these quantities would make it difficult to interpret a positive or negative aggregate signal or to defend a near-real-time performance claim.

1.3 Objectives and research questions

RQ1 asks how VADER, BiLSTM, RoBERTa and a simple linear baseline compare on the same labeled Twitter test examples. The associated objective is to report accuracy, class-level performance and uncertainty under a reproducible protocol.

RQ2 asks how model choice changes the sentiment distribution and time series for the same historical Bluesky posts. The associated objective is to calculate agreement, interval-based aggregation and trailing moving averages while retaining collection-coverage information.

RQ3 asks how local inference cost and offered arrival rate affect processing latency, queueing and completed records. The associated objective is to benchmark available inference implementations and replay a fixed workload through queueing and actual project storage.

1.4 Scope and contribution

The contribution is an empirical assessment of deployed model choices and temporal interpretation, supported by reproducible scripts and artifacts. It does not introduce a new neural architecture. The study evaluates a local implementation on a fixed benchmark and a two-week search corpus, with controlled short replay runs.

The proposal mentioned Twitter or Reddit and BERT. The implementation uses Bluesky and a Twitter RoBERTa checkpoint. These choices are recorded as implementation substitutions. RoBERTa is identified explicitly throughout the report. No claim of supervisor approval is made by this document.

2 Review of Related Literature

2.1 Lexicon and recurrent approaches

Hutto and Gilbert introduced VADER as a rule-based model for social-media sentiment [1]. A lexicon-based approach provides a useful computational baseline because it can score text without fitting a neural model in this project. Its predictions still need evaluation on the target task. Domain-specific vocabulary and context motivate examining its class-level errors.

LSTM networks were developed to address difficulties in learning long dependencies in recurrent models [2]. Bidirectional recurrent processing uses both sequence directions [3]. The project's BiLSTM adapts these ideas through a word-vectorization layer, learned embeddings and a three-class output. Its architecture alone provides no guarantee of better performance than a lexical or linear baseline. The local training procedure and class distribution must also be considered.

2.2 Transformers and domain-specific checkpoints

BERT uses bidirectional transformer pretraining to learn contextual representations [4]. RoBERTa studies how pretraining configuration and data influence the performance of BERT-style models [5]. These studies make training conditions a material part of any model comparison. The present experiment compares practical model instances with different pretraining histories, rather than attempting to isolate architecture as the causal source of performance differences.

TimeLMs provides Twitter language models that reflect different temporal training periods [7]. The Cardiff NLP sentiment checkpoint used here is documented as a Twitter-adapted model fine-tuned for sentiment [8]. Its exact cached revision is recorded in the experiment manifest. Our historical and benchmark scripts use the original local text with 128-token truncation. This implementation choice is reported because it may differ from other published preprocessing protocols.

2.3 Benchmarking and domain transfer

TweetEval provides a unified framework for evaluating several Twitter classification tasks, including sentiment [6]. It supplies the labeled source used for the local training, validation and test CSVs. A common test split allows direct comparison of predictions on the same examples. Macro recall is also reported because it is the sentiment metric used by TweetEval's evaluation script.

Applying a Twitter-trained sentiment model to Bluesky introduces a change in platform, time and topic distribution. This study has independent labels for the Twitter benchmark but no independent labels for the Bluesky sample. Therefore, benchmark ranking and Bluesky agreement answer different questions. The study can describe deployment outputs and their sensitivity, while local deployment correctness remains an open empirical question.

2.4 Existing Bluesky systems and temporal data

CognitiveSky describes a Bluesky analysis framework that combines API ingestion, transformer annotation and a dashboard for sentiment, emotion and narrative analysis [10]. It establishes direct overlap with the broad idea of building a Bluesky sentiment monitor. The present work concentrates on model disagreement, local timing and a bounded-queue replay rather than treating the platform or dashboard as the contribution.

BlueTempNet provides temporal multi-network data about Bluesky interactions, including user and community relationships [11]. Its focus differs from text polarity, but it demonstrates that temporal analysis of the platform already has a research context. The study reported here uses timestamped post-level sentiment predictions rather than an interaction-network representation.

Table 2.1 Relationship between selected prior work and this study

Work	**Relevant contribution**	**Role in this study**
VADER [1]	Rule-based social-media sentiment	Fast classification baseline
LSTM and bidirectionality [2, 3]	Recurrent sequence modeling	BiLSTM architecture foundation
BERT and RoBERTa [4, 5]	Contextual transformer pretraining	Interpretation of pretrained model comparison
TweetEval [6]	Common labeled evaluation tasks	Shared local test examples
TimeLMs [7]	Twitter models across time	Domain and temporal context
CognitiveSky [10]	Bluesky annotation and dashboard	Closest system-level overlap
BlueTempNet [11]	Temporal interaction networks	Related platform data research

2.5 Research gap addressed and its limits

This study tests whether accessible sentiment models change the interpretation of a common temporal corpus and measures their local processing costs. It links classification metrics, temporal outputs and controlled queueing evidence. This bounded empirical contribution does not establish first-of-kind novelty; publication positioning requires comparison with the closest full studies.

3 Research Methodology

3.1 Study design

The evaluation has three components. First, every model receives the same labeled Twitter test rows. Second, saved predictions for three models are aligned by URI and text against the cleaned Bluesky corpus. Third, available inference implementations are measured locally and under controlled arrivals. This design keeps the test questions explicit and makes each reported denominator traceable.

The experiment files retain model labels, row identifiers, class supports and input hashes. The scripts reject mismatched saved prediction rows or inconsistent corpus coverage. No new human annotation was conducted for this study, and no model-generated label is described as a human judgment.

Table 3.1 Datasets and their roles

Dataset	**Rows**	**Role**
TweetEval local training	45,615	Training TF-IDF baseline and existing BiLSTM
TweetEval local validation	2,000	Existing BiLSTM validation
TweetEval local test	12,284	Common labeled model evaluation
Historical raw Bluesky	129,159	AI query collection snapshot
Historical cleaned Bluesky	127,741	Agreement and temporal analysis
Unlabeled annotation sample	300	Retained file; excluded from accuracy claims

3.2 Collection and sampling

The historical collector requested the literal query AI, English language and latest ordering in nonoverlapping 15-minute UTC windows. It retained returned posts with a URI and a creation timestamp inside the requested half-open interval, and deduplicated URIs. The collection spans 19 October through 1 November 2026. The log records 1,344 successful windows, of which 194 reached the 100-result cap.

The collector did not paginate. A successful request indicates an API response, while a cap-hit window signals possible truncation. The clean-data process removes missing fields, invalid times, duplicate URIs, empty trimmed text and trimmed text shorter than ten characters. It removes 1,418 rows from the raw export. Search matches define the sampling frame. The polarity of a matched post does not necessarily express a stance toward AI.

3.3 Model configurations

VADER uses compound-score thresholds of +0.05 and −0.05. The linear baseline uses TF-IDF with at most 50,000 features, unigrams and bigrams, sublinear term frequency, and logistic regression with a maximum of 1,000 iterations. It fits on the local training split only. These are fixed baseline settings, with no test-set tuning.

The existing BiLSTM training script specifies a 20,000-token vocabulary, sequences of 100 tokens, 64-dimensional embeddings, 64 LSTM units in each direction, dropout 0.3, a 32-unit dense layer and a three-way softmax. Adam optimizes sparse categorical cross-entropy in batches of 64. Early stopping monitors validation loss with patience two and a maximum of ten epochs. The original realized training history and seed are unavailable, so the report describes the script configuration and scores the saved test predictions.

RoBERTa uses cardiffnlp/twitter-roberta-base-sentiment-latest at revision 3216a57f2a0d9c45a2e6c20157c20c49fb4bf9c7. Fresh benchmark inference uses a batch size of 32, a maximum length of 128 tokens, CPU execution and four PyTorch threads. VADER and RoBERTa are off-the-shelf models, whereas the linear baseline is fitted locally. The comparison does not match pretraining data or training budgets.

3.4 Metrics and uncertainty

Accuracy is the number of correct predictions divided by the number of test examples. For each class, precision divides true positives by predicted positives and recall divides true positives by actual positives. F1 is twice precision times recall divided by their sum, with zero assigned when its denominator is zero. Macro F1 and macro recall average the three class-specific values equally.

The uncertainty analysis uses 1,000 paired bootstrap resamples of test rows with seed 42. The same resampled row indices are used across models, and the 2.5th and 97.5th percentiles form a 95% interval. These intervals condition on the observed test set and assume independent rows. They do not quantify variation over retraining seeds, authors, future periods or platforms.

Normalized-text overlap is checked by lowercasing and collapsing whitespace. The local train/validation, train/test and validation/test intersections are all empty. This check does not rule out semantic duplicates, shared authors or exposure through a pretrained model’s corpus.

3.5 Temporal analysis

For each model and interval t, let P(t), N(t) and U(t) denote positive, negative and neutral counts. The total n(t) is their sum. Net sentiment $S(t) = [P(t) - N(t)] / n(t)$, ranging from −1 to +1 when observations exist. Empty intervals retain zero counts and undefined shares. The analysis uses creation time in UTC and produces 15-minute, hourly and daily series.

A trailing k-hour mean is the arithmetic mean of the last k hourly net-sentiment values, including the current period, with k equal to three or six. All k observations are required. This gives each observed hour equal weight. Daily net sentiment is separately calculated from daily counts and therefore weights posts. The analysis compares raw hourly series through Pearson correlation, mean absolute differences and opposite-sign periods.

Coverage sensitivity excludes hourly periods containing a capped request or missing/failed scheduled requests. The field fully_observed_uncapped in the CSV means that logged requests succeeded below the cap. It does not certify that the API returned every eligible post. Exclusion also changes the sample of hours.

3.6 Runtime and replay protocols

The warm inference benchmark samples 96 labeled test posts with seed 42. It measures tokenization and classification for VADER and RoBERTa, with three warm-up calls and three repeats at batch sizes one, eight and 32. Initialization, arrivals, queueing and storage are excluded. Batch-32 percentiles have only three observations per repeat and are descriptive.

Controlled replay samples 96 cleaned Bluesky posts with seed 42, one worker, batch size one, queue capacity 16 and offered rates of 10, 40 and 80 posts per second. Each condition runs three times. A full queue drops new arrivals. Admitted posts are classified and committed through db.insert_record into temporary SQLite storage. Monotonic timestamps record scheduled and actual arrivals, inference start/end and storage completion. Arrival-to-commit latency includes queueing and storage; 1,000 ms is a descriptive deadline. Each run reconciles offered, processed, dropped and stored counts. Replay excludes networking, filtering, collector scheduling, reconnects and backups; this finite workload does not establish sustained production capacity.

4 System Implementation

4.1 Data paths

The live-monitor path connects to Bluesky Jetstream, extracts post text, applies filtering, classifies accepted text with RoBERTa and stores a timestamped result in SQLite. Its filters cover junk text, known bot patterns, language, author posting rate and normalized duplicates. The confidence gate is disabled by default. Stored timestamps refer to processing, which differs from historical post-creation time.

The historical path collects search results and preserves URI, creation timestamp, collection timestamp, text and query. Separate scripts classify the same cleaned corpus with VADER, RoBERTa and BiLSTM. research_results.py aligns those predictions and exports agreement statistics. temporal_analysis.py adds coverage flags, moving averages and temporal comparisons. The benchmark path uses labeled Twitter files and does not merge them into the historical series.

Table 4.1 Implementation components

Component	**Responsibility**
stream_monitor.py	Live extraction, filtering, inference and orchestration
db.py	SQLite initialization, inserts and online backups
aggregate.py	Live database aggregation and exports
sentiment/benchmark.py	Common labeled evaluation and saved predictions
sentiment/benchmark_summary.py	Paired bootstrap intervals and overlap audit
sentiment/temporal_analysis.py	Historical temporal metrics and coverage
sentiment/runtime_benchmark.py	Repeated warm inference measurements
sentiment/replay_benchmark.py	Controlled arrivals, queueing and actual storage

4.2 Storage and reproducibility

The project database stores timestamp, text, sentiment and confidence score for each classified post. Each insert opens a connection, writes the record, commits and closes the connection. The replay harness reuses this function with temporary database paths so the experiment does not append test traffic to the collected dataset.

The experiment manifest records SHA-256 fingerprints for the source CSVs and BiLSTM model file. The test predictions retain original row IDs. Runtime and replay protocol files record sample selection and model revision. This allows a reader to distinguish rescoring stored predictions from running fresh model inference.

The inspected environment uses Python 3.12.7, PyTorch 2.4.1 CPU and Transformers 4.44.2 on Windows. Windows application control blocks a TensorFlow component even outside the execution sandbox. Therefore fresh BiLSTM inference and latency are unavailable in this environment. Its saved predictions remain usable for the row-checked test evaluation. The limitation is retained in all model-runtime comparisons.

4.3 Verification

The scripts verify that all three historical model outputs cover identical cleaned post IDs and text, and that BiLSTM saved test rows match the labeled test file. Class-count totals reconcile to 127,741 for each historical model across all three aggregation frequencies. Rolling means have the expected leading missing periods and match direct trailing-window calculations.

A fresh VADER run reproduced every saved historical VADER label. The SQLite quick check passed on the collected database. The database contained 64,853 rows at inspection, whereas an older raw CSV export contained 60,032. This difference is documented as snapshot age, and the study uses the named historical files for model comparisons.

5 Results and Discussion

5.1 Labeled benchmark performance

Table 5.1 reports the same 12,284 labeled test examples for every model. The neutral baseline, selected from the most frequent training label, reaches 48.33% accuracy. RoBERTa has the highest observed accuracy, macro F1 and macro recall among the evaluated model instances. These measurements concern the Twitter benchmark.

Table 5.1 Common labeled test performance and bootstrap accuracy intervals

Model	Accuracy	Macro F1	Macro recall	95% accuracy interval
VADER	53.00%	0.529	0.570	52.15%–53.93%
TF-IDF + LR	59.22%	0.560	0.567	58.37%–60.10%
BiLSTM	59.47%	0.558	0.558	58.61%–60.36%
RoBERTa	72.18%	0.724	0.735	71.38%–72.95%

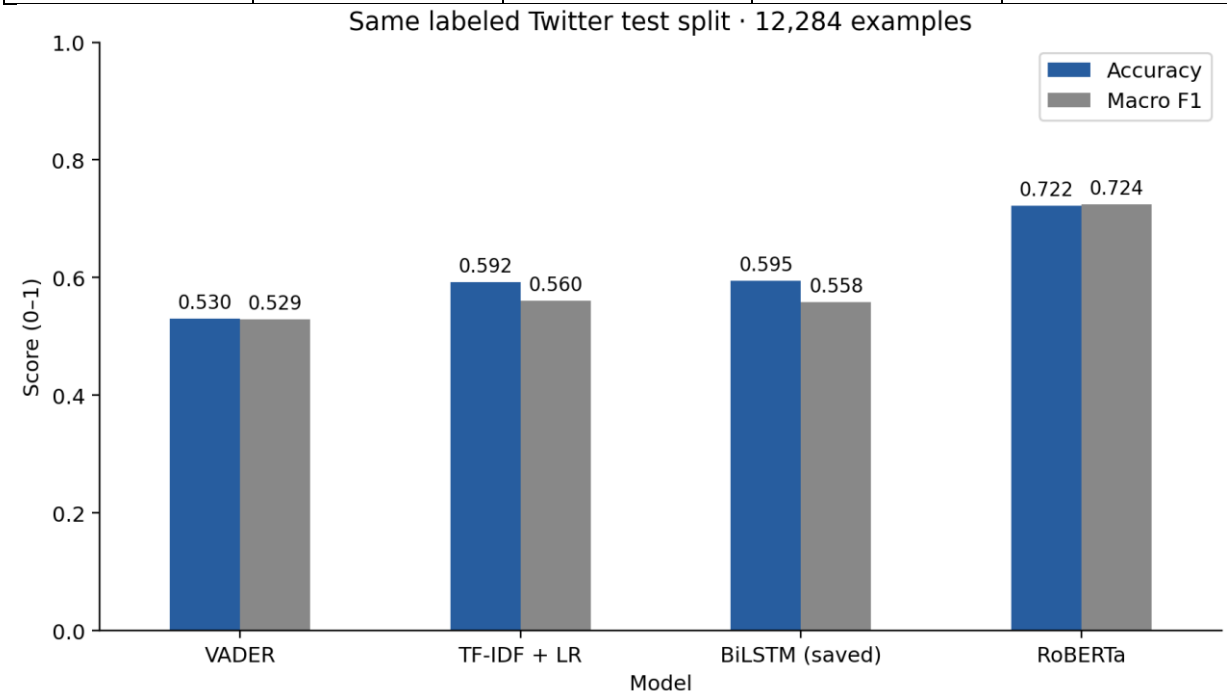


Figure 5.1 Accuracy and macro F1 on the local TweetEval test split. Source — benchmark prediction files.

5.2 Errors and paired comparisons

RoBERTa's negative recall is 0.807, neutral recall is 0.658 and positive recall is 0.741. Its confusion matrix contains 1,372 neutral examples predicted negative, showing that its higher aggregate score still coexists with substantial class-specific errors. The BiLSTM predicts neutral for 2,397 of 3,972 negative examples and achieves negative recall of 0.325.

The paired bootstrap interval for the linear baseline minus BiLSTM accuracy is approximately −0.0091 to +0.0042. Its macro-F1 difference interval is approximately −0.0057 to +0.0096. Both include zero, so the observed small differences do not support a clear ranking of these two systems under this resampling procedure.

The paired interval for BiLSTM minus RoBERTa accuracy is approximately −0.1377 to −0.1166. The corresponding macro-F1 interval is approximately −0.1762 to −0.1549. This supports an advantage for the evaluated RoBERTa checkpoint on these rows. It does not attribute that advantage solely to architecture or establish the same ranking on Bluesky.

Table 5.2 RoBERTa confusion matrix

Actual / predicted	**Negative**	**Neutral**	**Positive**
Negative	3204	711	57
Neutral	1372	3904	661
Positive	71	545	1759

5.3 Historical sentiment and agreement

The three models assign substantially different class distributions to the same search corpus. VADER labels 42.42% positive, compared with 11.44% for RoBERTa and 13.80% for BiLSTM. BiLSTM labels 70.27% neutral. These are model assignments over a query-defined sample, with unknown local correctness.

Table 5.3 Historical class assignments

Model	**Negative**	**Neutral**	**Positive**
VADER	41,765	31,790	54,186
RoBERTa	46,314	66,819	14,608
BiLSTM	20,349	89,768	17,624

Table 5.4 Historical pairwise agreement

Pair	**Agreement**	**Cohen κ**
VADER / RoBERTa	50.02%	0.289
VADER / BiLSTM	43.46%	0.209
RoBERTa / BiLSTM	65.31%	0.379

All three models agree on 42,830 posts, or 33.53% of the corpus. RoBERTa and BiLSTM agree most often at 65.31%, with kappa 0.379. Agreement identifies consistency between predictors, while a human-labeled target sample would be needed to establish correctness.

5.4 Temporal interpretation and coverage

VADER and RoBERTa give opposite net-sentiment signs in 327 of 336 hourly periods. Their raw hourly Pearson correlation is 0.460 and their mean absolute net-sentiment difference is 0.346. Positive correlation can coexist with opposite signs because the models can move together around different baselines.

Excluding capped hours retains 183 hourly periods. The VADER–RoBERTa mean absolute difference is 0.351 in that subset. The discrepancy therefore persists under this particular coverage exclusion. Every one of the fourteen days includes at least one cap-hit request, so there is no fully uncapped daily subset under the strict rule. No missing posts are imputed.

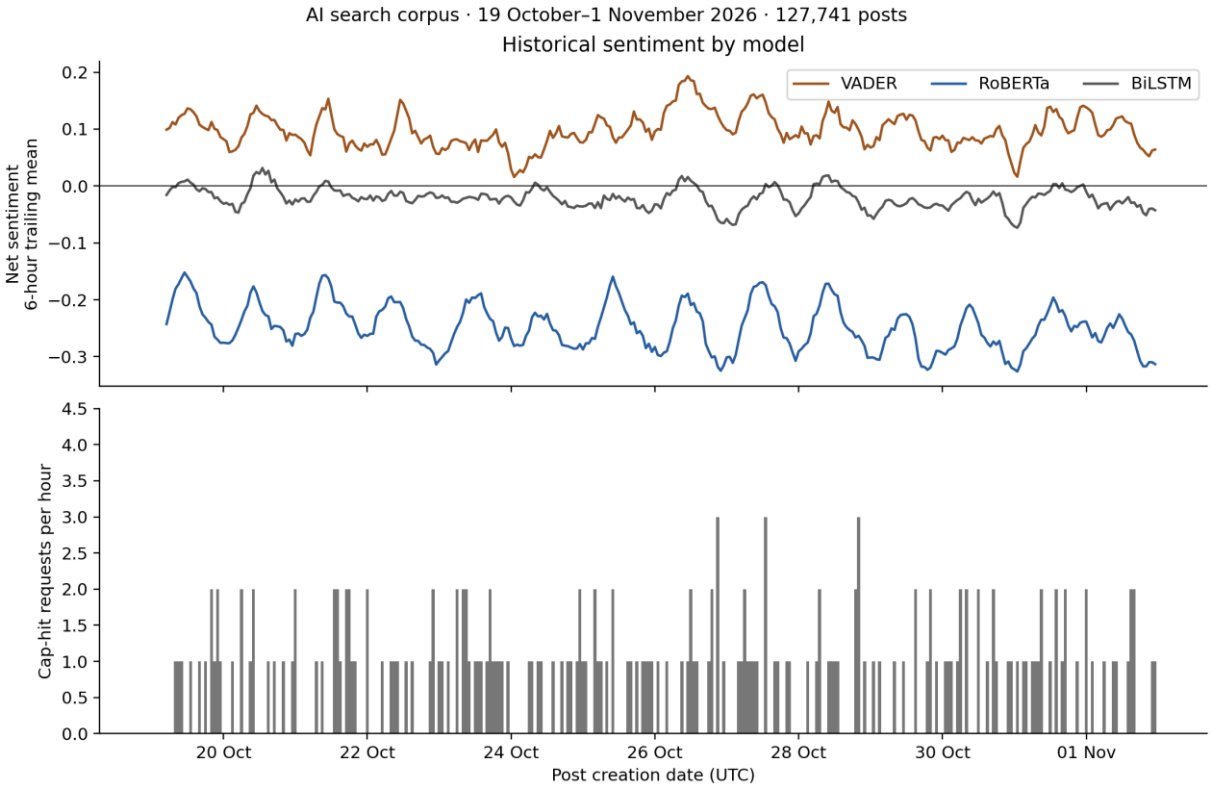


Figure 5.2 Six-hour trailing mean net sentiment and hourly cap-hit requests. Source — historical temporal exports, 19 October–1 November 2026 UTC.

Table 5.5 Sensitivity to aggregation interval for VADER and RoBERTa

Interval	Periods	Opposite signs	Mean absolute difference
15 minutes	1344	1114	0.346
Hourly	336	327	0.346
Daily	14	14	0.345

5.5 Warm inference measurements

For the fixed 96-post test sample, the median of the three RoBERTa batch-one p50 values is 26.45 ms, and the median p95 is 34.61 ms. The median batch-one throughput is 37.36 posts per

second. Batch eight reaches a median 59.60 posts per second on this short workload. These are local warm tokenization-plus-inference measurements.

VADER has much smaller inference costs on this workload. The absolute numbers are machine- and text-dependent and exclude storage. Repeated timing also shows run variation, particularly for very short calls. The subsequent replay experiment includes storage and uses longer historical search posts, so its latency and throughput need not match these inference-only measurements.

5.6 Controlled replay and overload

Table 5.6 Controlled replay over three runs per condition

Model	Offered posts/s	Stored of 288	Dropped of 288	Median run p95 ms
VADER	10	288	0	10.7
VADER	40	288	0	10.4
VADER	80	288	0	8.7
RoBERTa	10	191	97	3837.1
RoBERTa	40	79	209	4206.2
RoBERTa	80	72	216	3759.8

The table pools stored and dropped counts across three 96-post runs, while the latency column is the median of the three within-run p95 values for successful posts. Queue-overflow drops are excluded from those latency percentiles and are shown separately. A low latency percentile alone would therefore be an incomplete description of an overloaded system.

At the highest offered rate, RoBERTa drops 216 of 288 offered posts in this bounded-queue experiment. The result characterizes this worker, queue size, sample and storage path. Increasing capacity could postpone drops while increasing waiting time. Batching, parallel workers or alternate models would require new experiments before performance claims.

5.7 Answers to the research questions

For RQ1, the evaluated RoBERTa checkpoint leads the common labeled benchmark, while the linear baseline and saved BiLSTM have similar scores within the paired uncertainty intervals. For RQ2, classifier selection changes class proportions and often reverses the sign of the hourly net-sentiment series. For RQ3, local processing costs and queue capacity interact with offered rate, and controlled replay reveals behaviours that an inference-only timing cannot describe.

6 Conclusion and Future Scope

6.1 Conclusion

This project implements and evaluates a sentiment-monitoring workflow with a shared labeled benchmark, historical multi-model analysis and controlled local processing experiments. RoBERTa performs best on the labeled Twitter test set, but the deployment corpus shows large differences among model outputs. The linear baseline provides a competitive reference for the saved BiLSTM scores.

The temporal analysis demonstrates that a common input corpus can yield different aggregate sentiment directions depending on the classifier. The replay experiment adds evidence about queueing and storage under controlled offered rates. Reproducible scripts, saved predictions and raw timing records make the reported results inspectable.

6.2 Limitations

The historical sample covers one literal query and fourteen days. Capped windows, search ranking, filtering and unknown API recall constrain representativeness. Post polarity is not equivalent to stance toward AI. The lack of independent Bluesky labels prevents in-domain accuracy claims.

Benchmark results compare unequal pretraining histories and a single local training configuration. Bootstrap intervals assume independent rows and omit training-seed uncertainty. The saved BiLSTM model could not run in the current environment, so its scores are based on existing predictions and its timing is absent.

The controlled workloads are short and use fixed samples. Replay omits live network effects, filtering and collector orchestration. Tail latency is conditional on successful completion, and overloaded drops are reported separately. These tests do not establish long-term service reliability.

6.3 Future scope

Future work could obtain independent domain labels, expand queries and periods, and test longer production workloads. Routing between a fast model and RoBERTa requires thresholds selected on separate development data. Forecasting or anomaly detection requires chronological validation, baselines and coverage-aware evaluation. Publication development should compare the strongest empirical finding with closely related systems.

References

[1] C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text,” Proc. ICWSM, vol. 8, no. 1, pp. 216–225, 2014. doi: 10.1609/icwsm.v8i1.14550.

[2] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[3] M. Schuster and K. K. Paliwal, “Bidirectional Recurrent Neural Networks,” IEEE Transactions on Signal Processing, vol. 45, no. 11, pp. 2673–2681, 1997. doi: 10.1109/78.650093.

[4] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proc. NAACL-HLT, pp. 4171–4186, 2019. https://aclanthology.org/N19-1423/

[5] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv:1907.11692, 2019. https://arxiv.org/abs/1907.11692

[6] F. Barbieri, J. Camacho-Collados, L. Espinosa Anke and L. Neves, “TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification,” Findings of EMNLP, pp. 1644–1650, 2020. https://aclanthology.org/2020.findings-emnlp.148/

[7] D. Loureiro, F. Barbieri, L. Neves, L. Espinosa Anke and J. Camacho-Collados, “TimeLMs: Diachronic Language Models from Twitter,” Proc. ACL System Demonstrations, pp. 251–260, 2022. https://aclanthology.org/2022.acl-demo.25/

[8] Cardiff NLP, “twitter-roberta-base-sentiment-latest,” model card, accessed Nov. 9, 2026. Evaluated checkpoint revision 3216a57f2a0d9c45a2e6c20157c20c49fb4bf9c7. https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest

[9] M. Kleppmann et al., “Bluesky and the AT Protocol: Usable Decentralized Social Media,” DIN 2024. https://arxiv.org/abs/2402.03239

[10] G. Chhetri, A. Dutta and S. Das, “CognitiveSky: Scalable Sentiment and Narrative Analysis for Decentralized Social Media,” arXiv:2509.11444, 2025. https://arxiv.org/html/2509.11444v1

[11] U. Jeong, B. Jiang, Z. Tan, H. R. Bernard and H. Liu, “BlueTempNet: A Temporal Multi-network Dataset of Social Interactions in Bluesky Social,” arXiv:2407.17451, 2024. https://arxiv.org/abs/2407.17451

Appendix A Reproduction and Evidence

A.1 Execution order

Use the project Python 3.12 environment. Run sentiment/benchmark.py separately with --model vader, --model linear, --model bilstm_saved and --model roberta. Then run sentiment/benchmark_summary.py. Run sentiment/research_results.py and sentiment/temporal_analysis.py for historical outputs. Run sentiment/runtime_benchmark.py and sentiment/replay_benchmark.py sequentially after other inference jobs complete.

Run research/plot_results.py to regenerate figures. Raw predictions, confusion matrices, uncertainty intervals, runtime observations and replay summaries are included in the package. The README identifies commands and dependencies. The experiment manifest records the source fingerprints.

A.2 Report assembly notes

The institutional certificate, declaration, approval and signature pages must be supplied through the college process. They are excluded from this working submission copy as requested. The title page retains explicit fields for the learner’s full name, SAP number, guide, official department name and academic year. These details require confirmation before submission.

This document follows A4, Times New Roman 12 pt, 1.5 body spacing and the supplied 30/15/15/15 mm margins. Paragraph-after spacing is 30 pt as an interpretation of the guide’s 2.5-line gap. Main chapters start on fresh pages, use 18 pt centred headings, and use Arabic page numbering with the first Introduction page number hidden. Auxiliary pages use lower-case Roman numbers. The department should confirm the paragraph-gap interpretation.